

Interval width and bootstrap iterations: polytomous items, realistic misfit

Stylised 15-item and phq9-derived item parameters

Magnus Johansson

2026-08-02

Table of contents

Scope	1
Misfit strength calibrated against real data	1
Why the latent SD is held fixed	1
Two arms	2
Efficiency	2
Arm definitions	3
Run	4
Sidak relation at $k = 9$ and $k = 15$	300
Detection at realistic misfit strength	301
False positives on fitting items	303
Detection by targeting	304
FWER vs BH vs the width rule	306

Scope

Third instalment. `infit_width_iterations.qmd` used 20 dichotomous items; `infit_width_iterations_poly.qmd` repeated it with 20 polytomous items at an inter-dimensional correlation of 0.15. That confirmed the Sidak relation $(1 - w^k)$ for polytomous items but **saturated on detection** — every rule detected every misfitting item in every cell — so it could not price the rules against each other on power.

This run fixes that, and moves to conditions closer to applied practice.

Misfit strength calibrated against real data

Conditional infit on the real `phq9` responses ($N = 600$, $B = 1000$ cutoffs, FWER) flags three underfitting items: $q_3 = 1.234$, $q_8 = 1.260$, $q_9 = 1.315$. The earlier runs at $\rho = 0.15$ injected infit around 2.0 — roughly 60 percent larger than the worst-fitting real item, which is why detection saturated. Matching the real magnitude needs ρ near 0.85:

ρ	injected infit, $k = 1$	comparable to
0.75	1.38	worse than q_9 (1.32)
0.85	1.25	q_3 (1.23), q_8 (1.26)
0.90	1.17	below the flagged items

$\rho = 0.50$ (infit 1.71) is dropped — it exceeds anything observed in `phq9` and saturated in the previous run.

Why the latent SD is held fixed

Misfit magnitude is $\sigma * \sqrt{2 * (1 - \rho)}$, so the person SD and the inter-dimensional correlation are near-substitutes: $\sigma = 1.0$, $\rho = 0.50$ (misfit SD 1.00, infit 1.41) and $\sigma = 1.39$, $\rho = 0.75$ (misfit SD 0.98, infit 1.38) are practically the same condition. ρ is the misfit-strength knob here because σ additionally changes targeting and test information, making it a confounded lever. Each arm keeps its own justified SD: **1.39 for `phq9`** (estimated from the same responses as its item parameters) and **1.5 for `sim15`**. Varying targeting or reliability deliberately would deserve its own designed factor.

Two arms

	sim15	phq9
Items	15 stylised	9, CML from real responses
Locations	<code>seq(-2, 2, length.out = 15)</code>	clustered, range 2.46 logits
Thresholds	location +/- 1 logit, ordered	real; includes disordered (q9)
Latent SD	1.5	1.39 (estimated)
Misfitting items	1 or 3, at 0 / -1.14 / -2 logits	1 or 2: q5 (-0.15) then q8 (+1.40)
Sample sizes	250, 1000	150 , 250, 1000

n = 150 is used for the phq9 arm only. 15 polytomous items imply 45 threshold parameters, so at n = 150 the bootstrap replicates are sparse enough that `psychotools::pcmodel()` intermittently fails to invert the Hessian. The 9-item phq9 arm has 27 parameters and is the more appropriate vehicle for the small-sample condition anyway — a short scale with a small sample being the realistic pairing.

15 items rather than 20: since `rho` changes anyway, the comparison with the dichotomous study is already broken, and 5-15 items is the typical range for a unidimensional scale. Locations are evenly spaced so the 0 / -1 / -2 logit targeting design is exact rather than a property of a particular random draw.

For phq9, the misfitting items are **q5 and q8**, chosen for targeting: q5 sits essentially on the person mean (-0.15) and q8 is the most off-target item (+1.40). This prioritises the targeting contrast over using the items that happen to misfit in the real data — q3 (-0.40) would have been nearly as off-target as q8 in the opposite direction, giving no on-target case. Note that q5 fits well in the real data (infit 1.069, ns); what is borrowed from phq9 is its location and threshold structure, not its misfit.

Item counts of 9 and 15 give two further tests of $1 - w^k$, alongside $k = 20$ from the earlier studies.

Efficiency

`rho` only matters when misfitting items exist, so the **k = 0 condition is run once per arm** rather than once per `rho`. Cells are ordered so that the calibrated `rho = 0.85` condition and the cheaper sample sizes complete first — if the run is stopped early, the most informative cells are already done.

```
suppressMessages({
  source("../R/helpers.R") # package internals and shared helper functions
  library(ggdist)
  library(MASS)
  library(parallel)
  library(dplyr)
  library(tidyr)
  library(ggplot2)
})

WIDTHS <- c(0.90, 0.95, 0.99, 0.995, 0.999)
NS      <- c(150, 250, 1000)
BS      <- c(100, 400, 2000)
RHOS    <- c(0.85, 0.90, 0.75) # priority order; 0.85 matches real phq9 misfit
R_NULL  <- 120                 # raised from 60 on 2026-07-30; reps 61-120 resume
R_MIS   <- 120
NCORES  <- 10
ALPHA   <- 0.05
CHUNK   <- 60
RES     <- "infit_width_iterations_poly_weak.rds"
```

```
# run_resumable() is defined in ../R/helpers.R
```

Arm definitions

```
# ---- sim15: stylised, evenly spaced -----
S15_LOCS <- seq(-2, 2, length.out = 15)
S15_THR <- lapply(S15_LOCS, function(L) L + seq(-1, 1, length.out = 3))
S15_MIS <- vapply(c(0, -1, -2), function(tg) which.min(abs(S15_LOCS - tg)),
  integer(1))

# ---- phq9: CML estimates from the packaged real data -----
ph <- load_phq9_arm()
PH_THR <- ph$thr
PH_LOCS <- ph$locs
PH_SD <- ph$sd
PH_MIS <- c(5L, 8L) # on-target (-0.15) then most off-target (+1.40)

ARMS <- list(
  sim15 = list(thr = S15_THR, sd = 1.5, mis = S15_MIS, locs = S15_LOCS,
    nm = paste0("V", 1:15), ks = c(1L, 3L),
    ns = c(250, 1000)), # no n = 150: see note above
  phq9 = list(thr = PH_THR, sd = PH_SD, mis = PH_MIS, locs = PH_LOCS,
    nm = paste0("q", 1:9), ks = c(1L, 2L),
    ns = c(150, 250, 1000))
)

bind_rows(lapply(names(ARMS), function(a) {
  x <- ARMS[[a]]
  data.frame(arm = a, items = length(x$thr),
    misfit_items = paste(x$nm[x$mis], collapse = ", "),
    misfit_loc = paste(round(x$locs[x$mis], 2), collapse = ", "),
    latent_sd = round(x$sd, 2),
    k_levels = paste(x$ks, collapse = ", "),
    n_levels = paste(x$ns, collapse = ", "))
}))
```

arm	items	misfit_items	misfit_loc	latent_sd	k_levels	n_levels
sim15	15	V8, V4, V1	0, -1.14, -2	1.50	1, 3	250, 1000
phq9	9	q5, q8	-0.15, 1.4	1.39	1, 2	150, 250, 1000

```
sim_arm <- function(arm, n, k, rho, seed) {
  a <- ARMS[[arm]]
  set.seed(seed)
  S <- matrix(c(a$sd^2, rho * a$sd^2, rho * a$sd^2, a$sd^2), 2, 2)
  th <- MASS::mvrnorm(n, mu = c(0, 0), Sigma = S)
  ni <- length(a$thr)
  mis <- a$mis[seq_len(k)]
  d <- matrix(0L, n, ni)
  fit_items <- setdiff(seq_len(ni), mis)
  if (length(fit_items)) d[, fit_items] <- sim_partial_score(a$thr[fit_items], th[, 1])
  if (length(mis)) d[, mis] <- sim_partial_score(a$thr[mis], th[, 2])
  d <- as.data.frame(d)
  names(d) <- a$nm
  d
}

# bounds_at() and flag_code() are defined in ../R/helpers.R
```

Run

```
one_rep <- function(g) {
  out <- try({
    arm <- as.character(g$arm); rep <- g$rep; k <- g$k; n <- g$n; rho <- g$rho
    d <- sim_arm(arm, n, k, rho,
      seed = 9e7 + 1e7 * match(arm, names(ARMS)) + 1e6 * k +
      1e4 * round(rho * 100) + 1e3 * n + rep)
    items <- names(d)
    obs <- RMitemInfit(d, output = "dataframe")
    msq <- obs$Infit_MSQ[match(items, obs$Item)]
    hd <- list(); pv <- list()
    for (j in seq_along(BS)) {
      co <- try(RMitemInfitCutoff(d, iterations = BS[j], parallel = FALSE,
        seed = rep * 100 + j, verbose = FALSE),
        silent = TRUE)
      if (inherits(co, "try-error")) next
      hd[[length(hd) + 1L]] <- do.call(rbind, lapply(WIDTHS, function(w) {
        b <- bounds_at(co$results, w, items)
        data.frame(B = BS[j], w = w, item = seq_along(items),
          code = flag_code(msq, b[1, ], b[2, ]))
      })))
      fw <- try(suppressWarnings(
        RMitemInfit(d, cutoff = co, p_value = TRUE, correction = "fwer",
          alpha = ALPHA, output = "dataframe")), silent = TRUE)
      bh <- try(suppressWarnings(
        RMitemInfit(d, cutoff = co, p_value = TRUE, correction = "fdr_bh",
          alpha = ALPHA, output = "dataframe")), silent = TRUE)
      if (!inherits(fw, "try-error") && !inherits(bh, "try-error")) {
        i1 <- match(items, fw$Item); i2 <- match(items, bh$Item)
        pv[[length(pv) + 1L]] <- data.frame(
          B = BS[j], item = seq_along(items),
          p_marg = fw$p_infit[i1], padj_fwer = fw$padj_infit[i1],
          padj_bh = bh$padj_infit[i2])
      }
    }
    list(hdci = do.call(rbind, hd), pval = do.call(rbind, pv))
  }, silent = TRUE)
  if (inherits(out, "try-error")) NULL else out
}

# Grid in priority order: k = 0 first (all n ascending), then misfit blocks by
# rho (0.85 first), n ascending within each so cheap cells finish early.
blocks <- list()
for (a in names(ARMS)) {
  blocks[[length(blocks) + 1L]] <- expand.grid(
    rep = 1:R_NULL, n = ARMS[[a]]$ns, k = 0L, rho = 0, arm = a,
    stringsAsFactors = FALSE)[, c("arm", "rep", "k", "n", "rho")]
}
for (r in RHOS) for (a in names(ARMS)) {
  blocks[[length(blocks) + 1L]] <- expand.grid(
    rep = 1:R_MIS, n = ARMS[[a]]$ns, k = ARMS[[a]]$ks, rho = r, arm = a,
    stringsAsFactors = FALSE)[, c("arm", "rep", "k", "n", "rho")]
}
grid <- do.call(rbind, blocks)
grid <- grid[order(match(paste(grid$rho, grid$arm),
  unique(paste(grid$rho, grid$arm))), grid$n), ]
```

```
rownames(grid) <- NULL
cat("cells to run:", nrow(grid), "\n")
```

cells to run: 4200

```
s <- run_resumable(RES, grid, c("arm", "rep", "k", "n", "rho"), one_rep,
  extra = list(arms = lapply(ARMS, function(a)
    a[c("mis", "locs", "sd", "nm", "ks"))]))
```

```
cached: 4200 cells, nothing to run
```

```
bind_part <- function(part) {
  do.call(rbind, lapply(seq_len(nrow(s$grid)), function(i) {
    r <- s$res[[i]][[part]]
    if (is.null(r)) return(NULL)
    cbind(s$grid[i, c("arm", "rep", "k", "n", "rho")], r)
  })))
}

tag <- function(x) x |>
  mutate(is_misfit = mapply(function(arm, k, it)
    it %in% s$arms[[arm]]$mis[seq_len(k)], arm, k, item))
H <- bind_part("hdc1") |> tag() |>
  mutate(flag = as.integer(code > 0), w = sprintf("%.3f", w))
```

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]


```
Warning in data.frame(..., check.names = FALSE): row names were found from a
short variable and have been discarded
Warning in data.frame(..., check.names = FALSE): row names were found from a
short variable and have been discarded
Warning in data.frame(..., check.names = FALSE): row names were found from a
short variable and have been discarded
Warning in data.frame(..., check.names = FALSE): row names were found from a
short variable and have been discarded
Warning in data.frame(..., check.names = FALSE): row names were found from a
short variable and have been discarded
Warning in data.frame(..., check.names = FALSE): row names were found from a
short variable and have been discarded
Warning in data.frame(..., check.names = FALSE): row names were found from a
short variable and have been discarded
Warning in data.frame(..., check.names = FALSE): row names were found from a
short variable and have been discarded
Warning in data.frame(..., check.names = FALSE): row names were found from a
short variable and have been discarded
```

```
cat(nrow(s$grid), "cells complete |", round(s$elapsed_min, 1), "min\n")
```

```
4200 cells complete | 1108.4 min
```

Sidak relation at $k = 9$ and $k = 15$

```
expand.grid(w = WIDTHS, k = c(9, 15)) |>
  mutate(predicted = round(100 * (1 - w^k), 1)) |>
  pivot_wider(names_from = k, values_from = predicted, names_prefix = "k=") |>
  mutate(w = sprintf("%.3f", w))
```

w	k=9	k=15
0.900	61.3	79.4
0.950	37.0	53.7
0.990	8.6	14.0
0.995	4.4	7.2
0.999	0.9	1.5

Measured under a true model ($k = 0$):

```
H |>
  filter(k == 0) |>
  group_by(arm, w, B, n, rep) |>
  summarise(any = as.integer(sum(flag) > 0), .groups = "drop") |>
  group_by(arm, w, B, n) |>
  summarise(v = 100 * mean(any), .groups = "drop") |>
  pivot_wider(names_from = c(arm, n), values_from = v) |>
  mutate(across(where(is.numeric), \(x) round(x, 1))) |>
  arrange(w, B)
```

w	B	phq9_150	phq9_250	phq9_1000	sim15_250	sim15_1000
0.900	100	65.0	65.8	58.3	87.5	84.2
0.900	400	57.5	60.8	55.0	79.2	75.8
0.900	2000	56.7	57.5	55.0	75.8	76.7
0.950	100	45.0	45.8	40.8	61.7	56.7
0.950	400	35.8	40.0	30.0	52.5	59.2
0.950	2000	30.8	31.7	28.3	49.2	55.0
0.990	100	13.3	16.7	20.8	26.7	27.5
0.990	400	10.0	10.0	10.0	18.3	18.3
0.990	2000	5.8	9.2	7.5	10.8	15.8
0.995	100	11.7	15.0	18.3	22.5	20.0
0.995	400	8.3	8.3	10.0	10.8	10.8
0.995	2000	3.3	5.8	6.7	5.8	10.8
0.999	100	11.7	15.0	18.3	22.5	20.0
0.999	400	2.5	6.7	2.5	5.0	5.8
0.999	2000	0.0	0.8	1.7	1.7	1.7

Detection at realistic misfit strength

```
H |>
  filter(k > 0, is_misfit, B == 2000) |>
  group_by(arm, rho, k, w, n) |>
  summarise(det = 100 * mean(flag), .groups = "drop") |>
  pivot_wider(names_from = c(arm, n), values_from = det) |>
  mutate(across(where(is.numeric), \(x) round(x, 1))) |>
  arrange(rho, k, w)
```

rho	k	w	phq9_150	phq9_250	phq9_1000	sim15_250	sim15_1000
0.8	1	0.900	95.8	100.0	100.0	100.0	100.0
0.8	1	0.900	81.7	92.5	100.0	96.7	100.0
0.8	1	0.950	92.5	99.2	100.0	100.0	100.0
0.8	1	0.950	74.2	85.8	100.0	94.2	100.0
0.8	1	0.990	81.7	98.3	100.0	100.0	100.0
0.8	1	0.990	56.7	67.5	100.0	84.2	100.0
0.8	1	0.995	77.5	96.7	100.0	100.0	100.0
0.8	1	0.995	45.0	58.3	100.0	76.7	100.0
0.8	1	0.999	68.3	87.5	100.0	99.2	100.0
0.8	1	0.999	31.7	46.7	99.2	66.7	100.0
0.8	2	0.900	77.9	89.2	100.0	NA	NA
0.8	2	0.900	47.1	63.7	99.2	NA	NA

rho	k	w	phq9_150	phq9_250	phq9_1000	sim15_250	sim15_1000
0.8	2	0.950	67.9	80.4	100.0	NA	NA
0.8	2	0.950	36.7	50.8	97.1	NA	NA
0.8	2	0.990	47.1	66.2	99.6	NA	NA
0.8	2	0.990	20.4	33.8	88.3	NA	NA
0.8	2	0.995	37.9	60.8	99.6	NA	NA
0.8	2	0.995	14.2	29.2	85.0	NA	NA
0.8	2	0.999	23.8	48.8	98.8	NA	NA
0.8	2	0.999	6.2	15.0	77.9	NA	NA
0.8	3	0.900	NA	NA	NA	87.8	99.2
0.8	3	0.900	NA	NA	NA	64.4	93.1
0.8	3	0.950	NA	NA	NA	81.7	99.2
0.8	3	0.950	NA	NA	NA	51.1	90.3
0.8	3	0.990	NA	NA	NA	63.9	98.9
0.8	3	0.990	NA	NA	NA	33.1	80.6
0.8	3	0.995	NA	NA	NA	56.9	97.8
0.8	3	0.995	NA	NA	NA	27.5	77.2
0.8	3	0.999	NA	NA	NA	44.2	94.7
0.8	3	0.999	NA	NA	NA	16.9	65.3
0.9	1	0.900	54.2	67.5	99.2	77.5	100.0
0.9	1	0.950	42.5	55.0	98.3	69.2	100.0
0.9	1	0.990	20.0	35.8	96.7	50.0	100.0
0.9	1	0.995	11.7	30.8	93.3	40.0	99.2
0.9	1	0.999	7.5	18.3	85.0	23.3	98.3
0.9	2	0.900	30.0	47.1	81.7	NA	NA
0.9	2	0.950	20.8	32.1	75.0	NA	NA
0.9	2	0.990	7.1	15.4	56.7	NA	NA
0.9	2	0.995	2.9	9.6	50.8	NA	NA
0.9	2	0.999	0.8	6.2	39.2	NA	NA
0.9	3	0.900	NA	NA	NA	41.4	78.3
0.9	3	0.950	NA	NA	NA	31.1	71.4
0.9	3	0.990	NA	NA	NA	16.4	53.6
0.9	3	0.995	NA	NA	NA	10.8	44.4
0.9	3	0.999	NA	NA	NA	5.0	31.9

False positives on fitting items

```
H |>
  filter(k > 0, !is_misfit, B == 2000) |>
  group_by(arm, rho, k, w, n) |>
  summarise(fp = 100 * mean(flag), .groups = "drop") |>
  pivot_wider(names_from = c(arm, n), values_from = fp) |>
  mutate(across(where(is.numeric), \(x) round(x, 1))) |>
  arrange(rho, k, w)
```

rho	k	w	phq9_150	phq9_250	phq9_1000	sim15_250	sim15_1000
0.8	1	0.900	14.5	16.8	42.5	12.4	25.6
0.8	1	0.900	10.5	13.0	23.4	9.9	14.3
0.8	1	0.950	7.5	8.9	30.1	6.4	16.4
0.8	1	0.950	5.7	7.3	14.8	5.3	7.8
0.8	1	0.990	2.1	2.3	14.3	1.4	5.7
0.8	1	0.990	1.2	2.6	5.0	1.4	2.0
0.8	1	0.995	1.5	1.0	9.9	0.8	3.5
0.8	1	0.995	0.6	1.6	3.3	0.9	1.1
0.8	1	0.999	0.2	0.4	4.3	0.4	1.8
0.8	1	0.999	0.2	0.4	1.4	0.2	0.2
0.8	2	0.900	14.5	20.6	57.6	NA	NA
0.8	2	0.900	11.3	13.7	29.8	NA	NA
0.8	2	0.950	8.9	11.9	45.6	NA	NA
0.8	2	0.950	7.3	7.5	20.1	NA	NA
0.8	2	0.990	3.1	4.5	22.5	NA	NA
0.8	2	0.990	1.2	1.8	7.1	NA	NA
0.8	2	0.995	1.4	2.4	16.7	NA	NA
0.8	2	0.995	1.2	1.1	5.0	NA	NA
0.8	2	0.999	0.4	0.7	9.2	NA	NA
0.8	2	0.999	0.0	0.5	1.7	NA	NA
0.8	3	0.900	NA	NA	NA	19.9	48.8
0.8	3	0.900	NA	NA	NA	13.8	25.3
0.8	3	0.950	NA	NA	NA	11.6	35.6
0.8	3	0.950	NA	NA	NA	7.3	15.6
0.8	3	0.990	NA	NA	NA	3.2	16.9
0.8	3	0.990	NA	NA	NA	2.0	4.9
0.8	3	0.995	NA	NA	NA	1.7	13.0
0.8	3	0.995	NA	NA	NA	1.0	3.1
0.8	3	0.999	NA	NA	NA	0.7	6.0

rho	k	w	phq9_150	phq9_250	phq9_1000	sim15_250	sim15_1000
0.8	3	0.999	NA	NA	NA	0.3	1.2
0.9	1	0.900	10.1	12.0	14.6	9.8	11.4
0.9	1	0.950	5.0	7.2	8.3	4.6	5.5
0.9	1	0.990	1.5	2.0	2.4	1.2	1.7
0.9	1	0.995	0.8	1.1	1.8	0.6	1.1
0.9	1	0.999	0.2	0.4	0.7	0.2	0.4
0.9	2	0.900	9.9	12.1	18.2	NA	NA
0.9	2	0.950	5.1	6.2	11.5	NA	NA
0.9	2	0.990	1.3	1.4	3.0	NA	NA
0.9	2	0.995	0.6	0.6	1.7	NA	NA
0.9	2	0.999	0.4	0.1	0.7	NA	NA
0.9	3	0.900	NA	NA	NA	11.0	17.6
0.9	3	0.950	NA	NA	NA	6.0	9.4
0.9	3	0.990	NA	NA	NA	1.2	3.5
0.9	3	0.995	NA	NA	NA	0.7	2.0
0.9	3	0.999	NA	NA	NA	0.2	0.5

Detection by targeting

The misfitting items differ in targeting within each arm, so detection is broken out per item.

```
H |>
  filter(k > 0, is_misfit, B == 2000, w == "0.999") |>
  mutate(loc = round(mapply(function(a, i) s$arms[[a]]$locs[i], arm, item), 2)) |>
  group_by(arm, rho, k, n, item, loc) |>
  summarise(det = round(100 * mean(flag), 1), .groups = "drop") |>
  arrange(arm, rho, k, n, loc)
```

arm	rho	k	n	item	loc	det
phq9	0.75	1	150	5	-0.15	68.3
phq9	0.75	1	250	5	-0.15	87.5
phq9	0.75	1	1000	5	-0.15	100.0
phq9	0.75	2	150	5	-0.15	30.0
phq9	0.75	2	150	8	1.40	17.5
phq9	0.75	2	250	5	-0.15	69.2
phq9	0.75	2	250	8	1.40	28.3
phq9	0.75	2	1000	5	-0.15	100.0
phq9	0.75	2	1000	8	1.40	97.5
phq9	0.85	1	150	5	-0.15	31.7
phq9	0.85	1	250	5	-0.15	46.7

arm	rho	k	n	item	loc	det
phq9	0.85	1	1000	5	-0.15	99.2
phq9	0.85	2	150	5	-0.15	7.5
phq9	0.85	2	150	8	1.40	5.0
phq9	0.85	2	250	5	-0.15	23.3
phq9	0.85	2	250	8	1.40	6.7
phq9	0.85	2	1000	5	-0.15	91.7
phq9	0.85	2	1000	8	1.40	64.2
phq9	0.90	1	150	5	-0.15	7.5
phq9	0.90	1	250	5	-0.15	18.3
phq9	0.90	1	1000	5	-0.15	85.0
phq9	0.90	2	150	5	-0.15	0.0
phq9	0.90	2	150	8	1.40	1.7
phq9	0.90	2	250	5	-0.15	6.7
phq9	0.90	2	250	8	1.40	5.8
phq9	0.90	2	1000	5	-0.15	58.3
phq9	0.90	2	1000	8	1.40	20.0
sim15	0.75	1	250	8	0.00	99.2
sim15	0.75	1	1000	8	0.00	100.0
sim15	0.75	3	250	1	-2.00	10.8
sim15	0.75	3	250	4	-1.14	44.2
sim15	0.75	3	250	8	0.00	77.5
sim15	0.75	3	1000	1	-2.00	85.8
sim15	0.75	3	1000	4	-1.14	98.3
sim15	0.75	3	1000	8	0.00	100.0
sim15	0.85	1	250	8	0.00	66.7
sim15	0.85	1	1000	8	0.00	100.0
sim15	0.85	3	250	1	-2.00	6.7
sim15	0.85	3	250	4	-1.14	10.8
sim15	0.85	3	250	8	0.00	33.3
sim15	0.85	3	1000	1	-2.00	35.8
sim15	0.85	3	1000	4	-1.14	66.7
sim15	0.85	3	1000	8	0.00	93.3
sim15	0.90	1	250	8	0.00	23.3
sim15	0.90	1	1000	8	0.00	98.3
sim15	0.90	3	250	1	-2.00	0.8

arm	rho	k	n	item	loc	det
sim15	0.90	3	250	4	-1.14	3.3
sim15	0.90	3	250	8	0.00	10.8
sim15	0.90	3	1000	1	-2.00	5.0
sim15	0.90	3	1000	4	-1.14	32.5
sim15	0.90	3	1000	8	0.00	58.3

FWER vs BH vs the width rule

```
PL <- P |>
  mutate(fwer = as.integer(padj_fwer < ALPHA),
         bh = as.integer(padj_bh < ALPHA)) |>
  pivot_longer(c(fwer, bh), names_to = "rule", values_to = "flag")

bind_rows(
  H |> filter(k > 0, B == 2000, w == "0.999") |>
    group_by(arm, rho, k, n) |>
    summarise(rule = "HDCI w=.999",
              det = 100 * mean(flag[is_misfit]),
              fp = 100 * mean(flag[!is_misfit]), .groups = "drop"),
  PL |> filter(k > 0, B == 2000) |>
    group_by(arm, rho, k, n, rule) |>
    summarise(det = 100 * mean(flag[is_misfit], na.rm = TRUE),
              fp = 100 * mean(flag[!is_misfit], na.rm = TRUE), .groups = "drop")
) |>
  mutate(across(c(det, fp), \(x) round(x, 1))) |>
  arrange(arm, rho, k, n, rule)
```

arm	rho	k	n	rule	det	fp
phq9	0.75	1	150	HDCI w=.999	68.3	0.2
phq9	0.75	1	150	bh	80.8	1.2
phq9	0.75	1	150	fwer	80.0	0.1
phq9	0.75	1	250	HDCI w=.999	87.5	0.4
phq9	0.75	1	250	bh	97.5	1.1
phq9	0.75	1	250	fwer	97.5	0.5
phq9	0.75	1	1000	HDCI w=.999	100.0	4.3
phq9	0.75	1	1000	bh	100.0	15.9
phq9	0.75	1	1000	fwer	100.0	8.5
phq9	0.75	2	150	HDCI w=.999	23.8	0.4
phq9	0.75	2	150	bh	47.1	2.3
phq9	0.75	2	150	fwer	45.0	1.0
phq9	0.75	2	250	HDCI w=.999	48.8	0.7
phq9	0.75	2	250	bh	70.4	4.4
phq9	0.75	2	250	fwer	65.8	1.5

arm	rho	k	n	rule	det	fp
phq9	0.75	2	1000	HDCI w=,999	98.8	9.2
phq9	0.75	2	1000	bh	100.0	35.1
phq9	0.75	2	1000	fwer	99.6	18.2
phq9	0.85	1	150	HDCI w=,999	31.7	0.2
phq9	0.85	1	150	bh	49.2	0.5
phq9	0.85	1	150	fwer	50.0	0.3
phq9	0.85	1	250	HDCI w=,999	46.7	0.4
phq9	0.85	1	250	bh	62.5	1.8
phq9	0.85	1	250	fwer	62.5	1.0
phq9	0.85	1	1000	HDCI w=,999	99.2	1.4
phq9	0.85	1	1000	bh	100.0	5.2
phq9	0.85	1	1000	fwer	100.0	3.2
phq9	0.85	2	150	HDCI w=,999	6.2	0.0
phq9	0.85	2	150	bh	18.3	0.4
phq9	0.85	2	150	fwer	18.8	0.1
phq9	0.85	2	250	HDCI w=,999	15.0	0.5
phq9	0.85	2	250	bh	32.9	1.8
phq9	0.85	2	250	fwer	31.7	0.7
phq9	0.85	2	1000	HDCI w=,999	77.9	1.7
phq9	0.85	2	1000	bh	90.8	8.8
phq9	0.85	2	1000	fwer	86.7	4.8
phq9	0.90	1	150	HDCI w=,999	7.5	0.2
phq9	0.90	1	150	bh	15.0	0.7
phq9	0.90	1	150	fwer	15.0	0.5
phq9	0.90	1	250	HDCI w=,999	18.3	0.4
phq9	0.90	1	250	bh	32.5	1.0
phq9	0.90	1	250	fwer	33.3	0.6
phq9	0.90	1	1000	HDCI w=,999	85.0	0.7
phq9	0.90	1	1000	bh	95.0	2.4
phq9	0.90	1	1000	fwer	96.7	1.4
phq9	0.90	2	150	HDCI w=,999	0.8	0.4
phq9	0.90	2	150	bh	5.8	0.4
phq9	0.90	2	150	fwer	5.8	0.4
phq9	0.90	2	250	HDCI w=,999	6.2	0.1
phq9	0.90	2	250	bh	13.3	0.5

arm	rho	k	n	rule	det	fp
phq9	0.90	2	250	fwer	12.1	0.1
phq9	0.90	2	1000	HDCI w=.999	39.2	0.7
phq9	0.90	2	1000	bh	55.4	3.3
phq9	0.90	2	1000	fwer	53.3	1.5
sim15	0.75	1	250	HDCI w=.999	99.2	0.4
sim15	0.75	1	250	bh	100.0	0.4
sim15	0.75	1	250	fwer	100.0	0.2
sim15	0.75	1	1000	HDCI w=.999	100.0	1.8
sim15	0.75	1	1000	bh	100.0	3.6
sim15	0.75	1	1000	fwer	100.0	2.2
sim15	0.75	3	250	HDCI w=.999	44.2	0.7
sim15	0.75	3	250	bh	63.1	2.0
sim15	0.75	3	250	fwer	56.7	0.8
sim15	0.75	3	1000	HDCI w=.999	94.7	6.0
sim15	0.75	3	1000	bh	98.9	21.5
sim15	0.75	3	1000	fwer	98.1	9.9
sim15	0.85	1	250	HDCI w=.999	66.7	0.2
sim15	0.85	1	250	bh	76.7	0.5
sim15	0.85	1	250	fwer	75.0	0.4
sim15	0.85	1	1000	HDCI w=.999	100.0	0.2
sim15	0.85	1	1000	bh	100.0	0.9
sim15	0.85	1	1000	fwer	100.0	0.3
sim15	0.85	3	250	HDCI w=.999	16.9	0.3
sim15	0.85	3	250	bh	26.4	0.7
sim15	0.85	3	250	fwer	24.4	0.2
sim15	0.85	3	1000	HDCI w=.999	65.3	1.2
sim15	0.85	3	1000	bh	80.6	5.3
sim15	0.85	3	1000	fwer	77.2	2.1
sim15	0.90	1	250	HDCI w=.999	23.3	0.2
sim15	0.90	1	250	bh	32.5	0.4
sim15	0.90	1	250	fwer	38.3	0.3
sim15	0.90	1	1000	HDCI w=.999	98.3	0.4
sim15	0.90	1	1000	bh	99.2	1.0
sim15	0.90	1	1000	fwer	99.2	0.4
sim15	0.90	3	250	HDCI w=.999	5.0	0.2

arm	rho	k	n	rule	det	fp
sim15	0.90	3	250	bh	12.2	0.5
sim15	0.90	3	250	fwer	10.8	0.2
sim15	0.90	3	1000	HDCI w=.999	31.9	0.5
sim15	0.90	3	1000	bh	51.7	2.2
sim15	0.90	3	1000	fwer	43.3	1.0

```

bind_rows(
  H |> filter(k > 0, B == 2000, w == "0.999") |>
    group_by(arm, rho, k, n) |>
    summarise(rule = "HDCI .999", det = 100 * mean(flag[is_misfit]), .groups = "drop"),
  PL |> filter(k > 0, B == 2000) |>
    group_by(arm, rho, k, n, rule) |>
    summarise(det = 100 * mean(flag[is_misfit], na.rm = TRUE), .groups = "drop")
) |>
ggplot(aes(factor(rho), det, colour = rule, group = rule)) +
  geom_line() + geom_point() +
  facet_grid(arm + k ~ n, labeller = label_both) +
  labs(x = "Inter-dimensional correlation (higher = weaker misfit)",
       y = "Detection (%)", colour = "Rule") +
  theme_minimal(base_size = 10)

```

